Web Scraping Cheatsheet

requests · BeautifulSoup4 · Playwright · tqdm

1. Which tool for which site?

Situation	Tool	Why
Static HTML — data is in the raw response	`requests + BS4`	No JS execution needed
JS-rendered page — data loads via JavaScript	`Playwright`	Drives a real browser
ASP.NET dropdowns / AJAX postbacks (e.g. DWS)	`Playwright`	Must wait for re-render
JSON API endpoint visible in Network tab	`requests`	Skip HTML entirely
File download (.csv, .tif, .gz)	`requests stream`	stream=True + iter_content
Parsing a table from already-rendered HTML	`pd.read_html()`	Returns DataFrames directly

Tip for DWS: it's ASP.NET with a province dropdown that triggers a server-side postback — that's a JS-rendered page. Playwright is the correct choice.

2. requests — HTTP calls

GET a page

Call requests.get(url). Check response.status_code == 200 before using content. Access HTML with .text or raw bytes with .content.

Stream large files

Pass stream=True to avoid loading everything into memory. Iterate response.iter_content(chunk_size) and write chunks to disk.

Headers / User-Agent

Some servers block the default Python user-agent. Pass a headers dict with a realistic browser User-Agent string to appear human.

Timeout + retry

Always set timeout=30. Without it a hanging server blocks forever. Wrap in try/except and retry on failure.

Query params

Pass a params dict to .get() — requests builds the query string for you. Cleaner than formatting URLs by hand.

raise_for_status()

Call response.raise_for_status() after every request. Raises an exception on 4xx/5xx instead of silently returning bad data.

Warning: never hardcode API keys or session tokens. Use python-dotenv to load them from a .env file that is listed in .gitignore.

3. BeautifulSoup4 — HTML parsing

Parser choice

Always pass 'lxml' as the second argument to BeautifulSoup(). It is faster and more lenient than Python's built-in html.parser. Already in your requirements.

find vs find_all

find() returns the first match. find_all() returns a list of all matches. Both accept tag name, CSS class, id, or attribute filters.

CSS selectors

soup.select('table.data-table td') lets you use full CSS selector syntax — useful when structure is deeply nested or class-dependent.

Tables → pandas

For HTML tables, skip BS4 entirely and use pd.read_html(html_string). It returns a list of DataFrames, one per table. Much faster.

Extracting text

Use .get_text(strip=True) on any element. Avoid .text — it doesn't strip whitespace. Always check for None before calling methods.

Attributes

Access tag attributes like a dict: tag['href'] or safer with tag.get('href') — the latter returns None instead of raising on missing attributes.

4. Playwright — browser automation

Execution flow

1	2	3	4	5	6
Launch browser	**Open page**	**Interact**	**Wait for render**	**Extract HTML**	**Close**
`async_playwright() → chromium.launch()`	`browser.new_page() → page.goto(url)`	`select_option() / click() / fill()`	`wait_for_load_state ('networkidle')`	`page.content() → pd.read_html()`	`browser.close()`

Setup (one-time)

After pip install playwright, run: playwright install chromium. This downloads the browser binary separately. Don't skip this step.

Async vs sync

Use async_playwright with asyncio.run(). The async API is better for scraping multiple pages or provinces without blocking.

Waiting correctly

Use wait_for_load_state('networkidle') after interactions. 'load' fires too early on AJAX-heavy pages like DWS.

Selecting dropdowns

page.select_option(selector, value=...) triggers the ASP.NET postback. Confirm the selector by inspecting the live page — ids change.

Extracting the table

After the page re-renders, call page.content() to get the full HTML, then pass it to pd.read_html() to get DataFrames.

Headless mode

headless=True runs without a visible browser window — use this in production. Set headless=False during development to watch what the browser is doing.

5. tqdm — progress bars for long loops

Wrap any iterable

Wrap a list or range in tqdm(...) to get a live progress bar automatically. Works with for loops, file downloads, and list comprehensions.

File download progress

Pass total=int(response.headers.get('content-length', 0)) and unit='B', unit_scale=True to show a human-readable download progress bar.

Nested loops	Manual update
Use tqdm(leave=False) on inner loops so they disappear after completion, keeping output clean when iterating over provinces and months.	Call bar.update(n) inside a chunk loop to advance the bar by n bytes/items. Use as a context manager (with tqdm(...) as bar) for clean cleanup.

6. Common gotchas for this project

🔴	**Data leakage**	Never use future data to predict the past. TimeSeriesSplit only.
🟠	**Missing timeout**	requests without timeout= hangs forever on a slow server.
🟠	**Status codes**	Always call raise_for_status() — a 200 doesn't mean you got data.
🟠	**ASP.NET render**	wait_for_load_state('networkidle') after dropdown selection, not just 'load'.
🟠	**Column drift**	DWS table headers change. Print df.columns on first run and hardcode nothing.
🟢	**Rate limiting**	Add a short sleep between province requests to avoid getting blocked.
🟢	**Idempotent saves**	Check if a file already exists before downloading. Resume-safe scraper.